

Chapter 4: Feature Detection

Assoc. Prof. Dr.-Ing. Nguyễn Xuân Hà

Robotics Group, Dept. of Mechatronics, SME - HUST

ha.nguyenxuan@hust.edu.vn

<https://sites.google.com/site/tsnguyenxuanhabkhn/home>

Lecture 4: Programming exercise

Bài 1: Cho ảnh như hình dưới đây (ảnh 'C4_B1.png' trong thư mục data đi kèm).



Sử dụng phương pháp Harris để phát hiện các góc của hình này theo các bước sau:

- Bước 1: Tính đạo hàm ảnh I_x , I_y dựa vào bộ lọc Sobel
- Bước 2: Tính các ma trận I_x^2 , I_y^2 và $I_x I_y$
- Bước 3: Tính hệ số đáp ứng Harris cho từng điểm ảnh với cửa sổ quan sát là 5×5
- Bước 4: Chọn 250 điểm ảnh là cực đại địa phương trong khu vực là 5×5 với thuật toán Non-Maximum Suppression
- Bước 5: Đánh dấu các vị trí này trên điểm ảnh.

Thực hiện so sánh với câu lệnh có sẵn trong opencv.

Lecture 4: Programming exercise

Phương pháp Harris để phát hiện góc:

$$f_{Harris} = \det(H) - \kappa \times (\text{trace}(H))^2$$

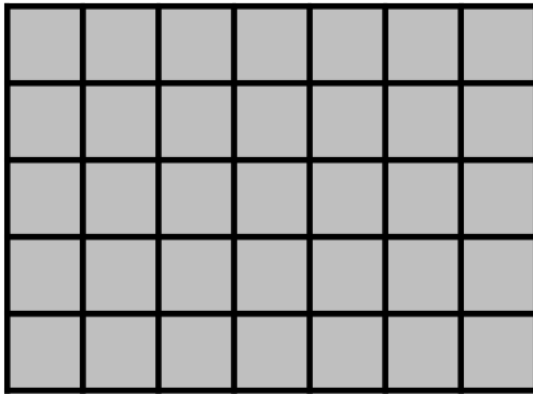
$$H = \begin{bmatrix} \sum_{(x,y) \in W} I_x^2 & \sum_{(x,y) \in W} I_x I_y \\ \sum_{(x,y) \in W} I_x I_y & \sum_{(x,y) \in W} I_y^2 \end{bmatrix} = \begin{bmatrix} h_{00} & h_{01} \\ h_{10} & h_{11} \end{bmatrix}$$

$$\det(H) = h_{00}h_{11} - h_{01}h_{10}$$

$$\text{trace}(H) = h_{00} + h_{11}$$

Lecture 4: Programming exercise

Phương pháp Harris để phát hiện góc:



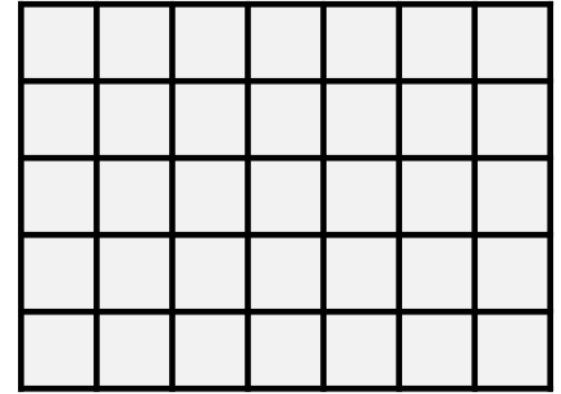
I

$*$

1	0	-1
2	0	-2
1	0	-1

S_x

$=$



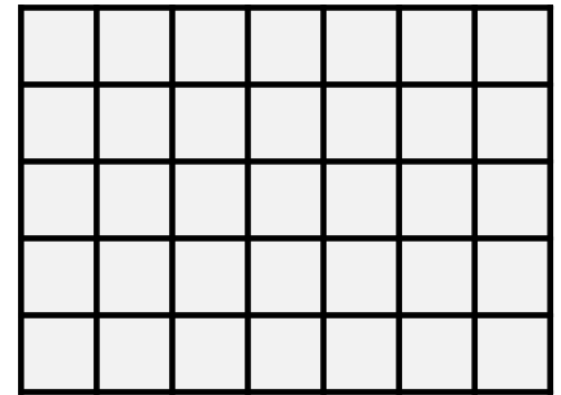
I_x

$*$

1	2	1
0	0	0
-1	-2	-1

S_y

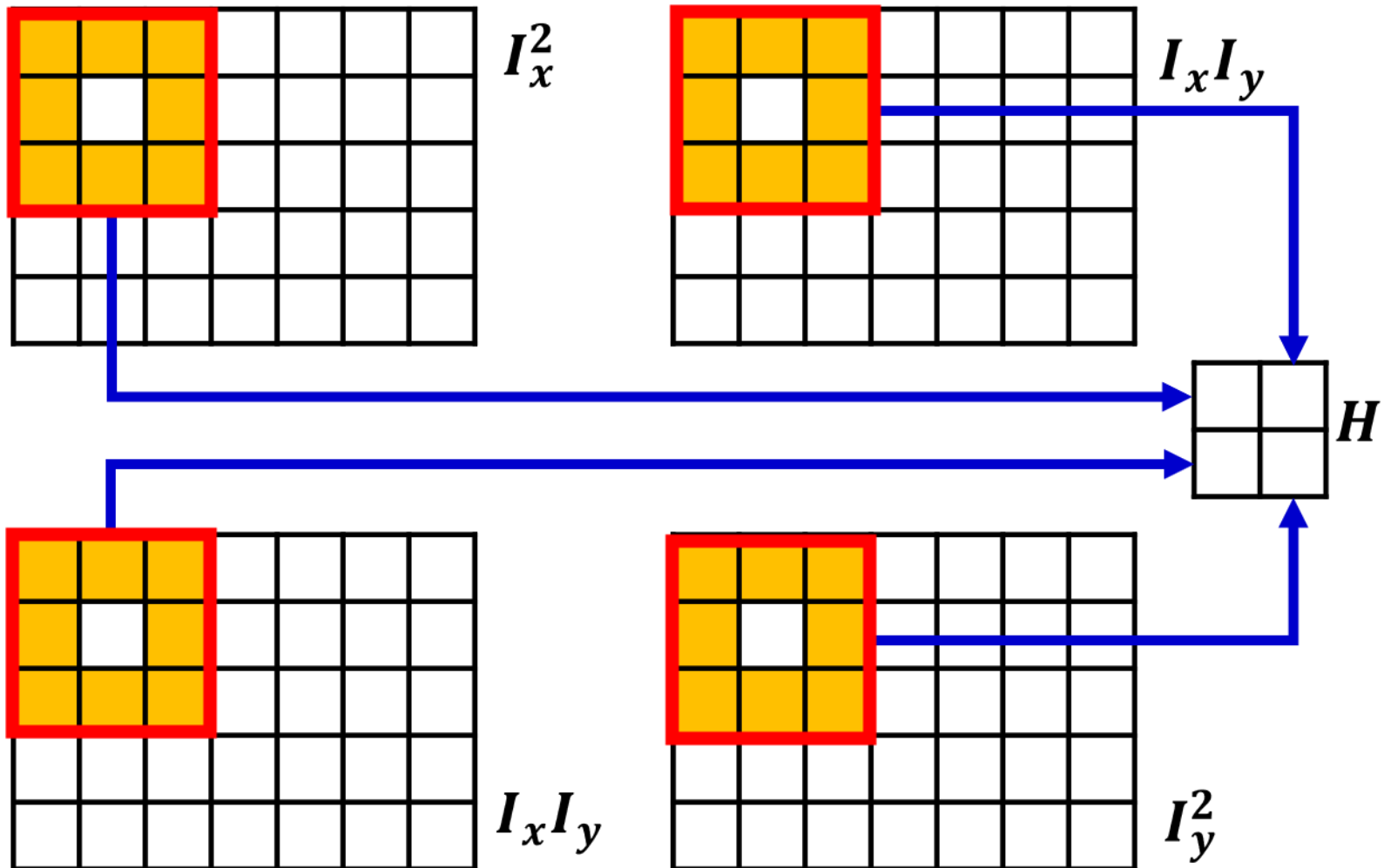
$=$



I_y

Lecture 4: Programming exercise

Phương pháp Harris để phát hiện góc:



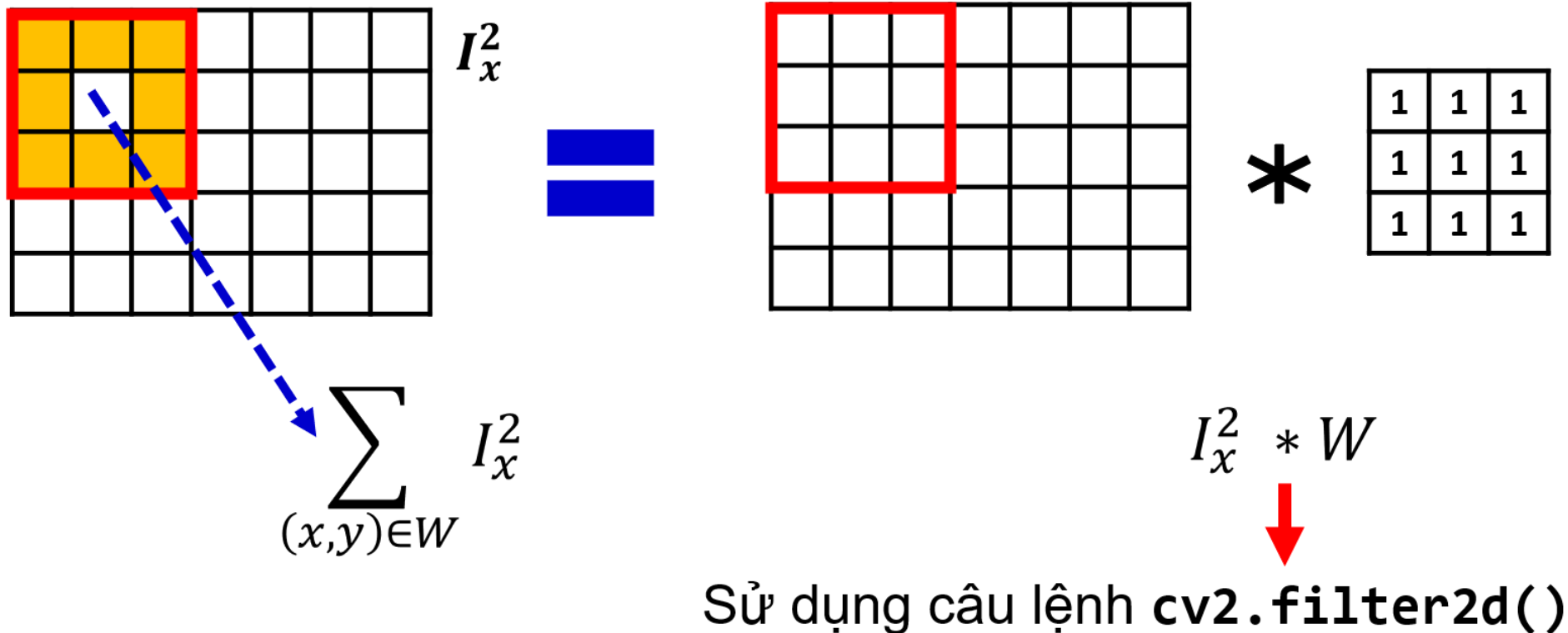
Lecture 4: Programming exercise

Phương pháp Harris để phát hiện góc: Quá trình trên được thực hiện cho tất cả các điểm ảnh

- Ưu điểm: dễ dàng thực hiện
- Nhược điểm: nếu tính toán lần lượt cho từng điểm ảnh một thì tốn rất nhiều thời gian → không hiệu quả
→ cần một thuật toán tính toán khác để xử lý nhanh hơn.

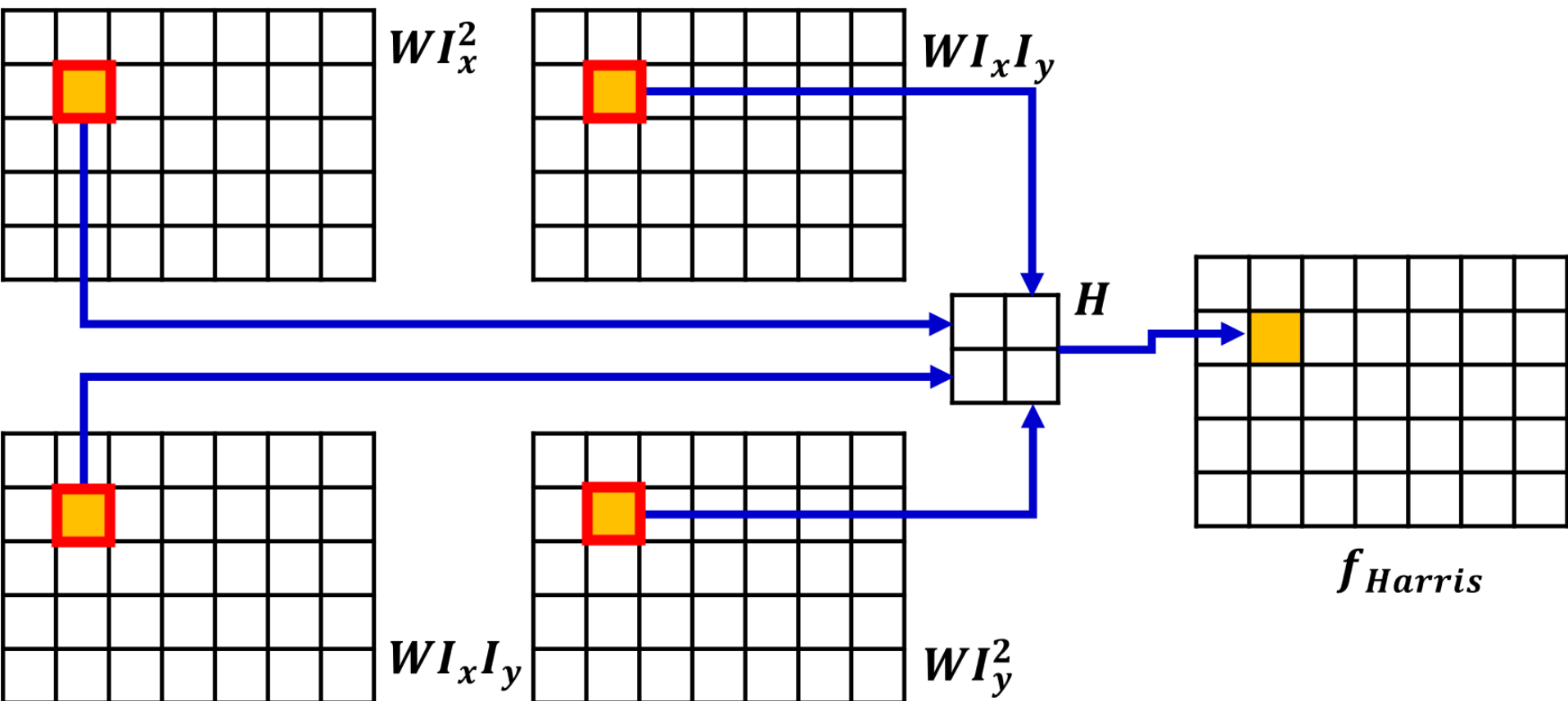
Lecture 4: Programming exercise

Phương pháp Harris để phát hiện góc



Lecture 4: Programming exercise

Phương pháp Harris để phát hiện góc



Lecture 4: Programming exercise

Phương pháp Harris để phát hiện góc

Đọc ảnh xám I

$$I_x = I * S_x; I_y = I * S_y$$

$$I_x^2 = I_x \otimes I_x; I_y^2 = I_y \otimes I_y; I_{xy} = I_x \otimes I_y$$

$$WI_x^2 = I_x^2 * W; WI_y^2 = I_y^2 * W; WI_{xy} = I_{xy} * W;$$

$$\begin{aligned} \det(H) &= WI_x^2 \otimes WI_y^2 - WI_{xy} \otimes WI_{xy}; \\ \text{trace}(H) &= WI_x^2 + WI_y^2 \end{aligned}$$

$$f_{\text{Harris}} = \det(H) - \kappa \times \text{trace}(H) \otimes \text{trace}(H)$$

Lecture 4: Programming exercise

Phương pháp Harris để phát hiện góc

```
def GetGradientImage(img):  
    img = img.astype(np.float64)  
    Sx = np.array([  
        [1, 0, -1],  
        [1, 0, -1],  
        [1, 0, -1]  
    ])  
    Sx = Sx / 3  
    Sy = np.array([  
        [1, 1, 1],  
        [0, 0, 0],  
        [-1, -1, -1]  
    ])  
    Sy = Sy / 3  
    Ix = cv2.filter2D(img, -1, Sx,  
                      borderType = cv2.BORDER_CONSTANT)  
    Iy = cv2.filter2D(img, -1, Sy,  
                      borderType = cv2.BORDER_CONSTANT)  
    return Ix, Iy
```

Áp dụng ngưỡng kép cho các giá trị

Tính các đạo hàm của bức ảnh

Lecture 4: Programming exercise

Phương pháp Harris để phát hiện góc

```
def CalculateHarrisResponse(img, window_size, kappa):  
    Ix, Iy = GetGradientImage(img)
```

```
    Ix2 = Ix * Ix  
    Iy2 = Iy * Iy  
    IxIy = Ix * Iy
```

Tính I_x^2 , I_y^2 và $I_x I_y$

```
    W = np.ones((window_size, window_size))  
    W_Ix2 = cv2.filter2D(Ix2, -1, W,  
                          borderType = cv2.BORDER_CONSTANT)  
    W_Iy2 = cv2.filter2D(Iy2, -1, W,  
                          borderType = cv2.BORDER_CONSTANT)  
    W_IxIy = cv2.filter2D(IxIy, -1, W,  
                          borderType = cv2.BORDER_CONSTANT)
```

Tính WI_x^2 , WI_y^2 và
 $WI_x I_y$

```
    det_H = W_Ix2 * W_Iy2 - W_IxIy * W_IxIy  
    trace_H = W_Ix2 + W_Iy2  
    f_Harris = det_H - kappa * trace_H * trace_H
```

Tính ma trận chứa
giá trị Harris cho
từng điểm pixel

```
    return f_Harris
```

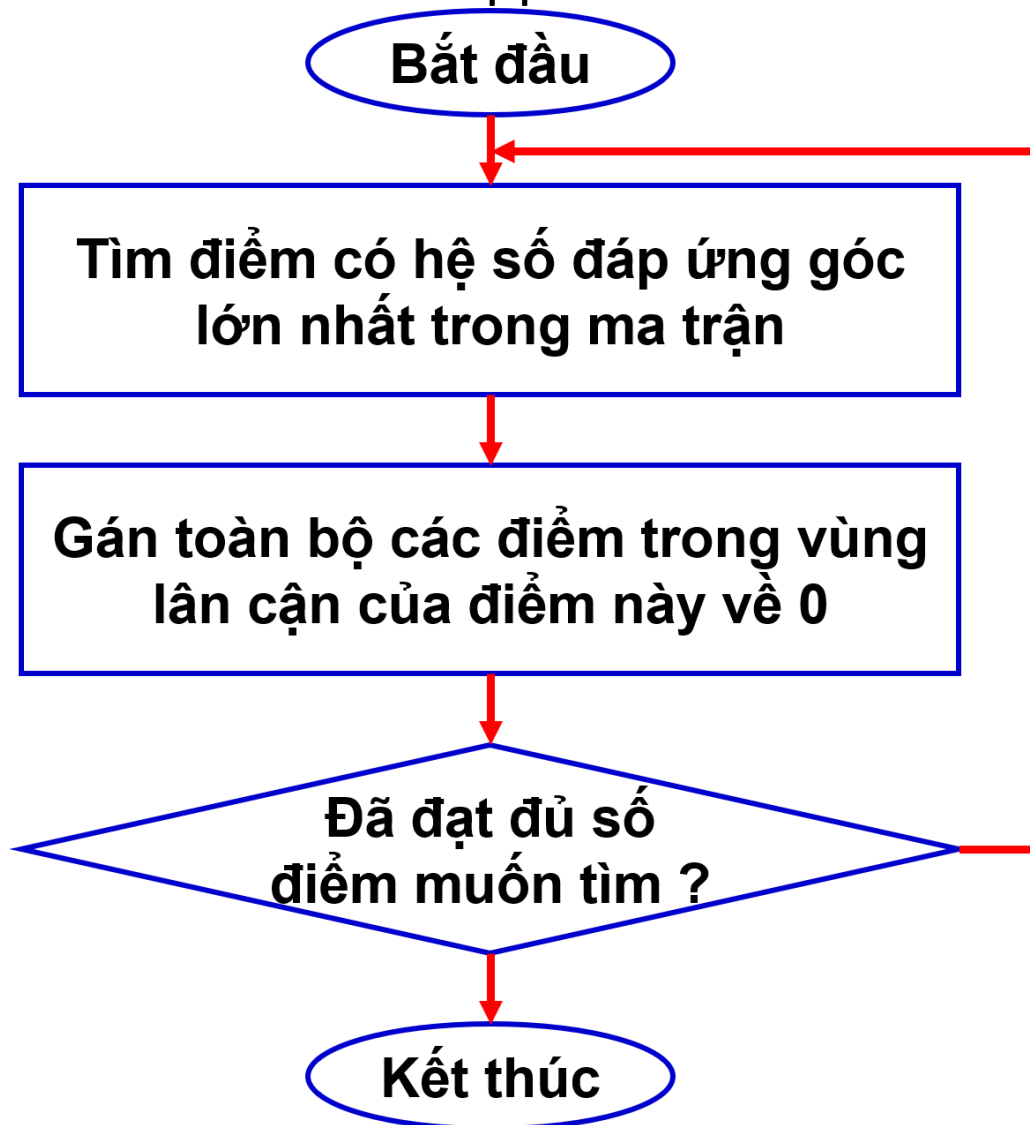
Lecture 4: Programming exercise

Kết quả tính toán hệ số Harris



Lecture 4: Programming exercise

Thuật toán Non-Maximum Suppression



Lecture 4: Programming exercise

Thuật toán Non-Maximum Suppression

```
def NonMaximumSuppression(f_Harris, kernel_size, num_keypoints):
```

```
    k = int ((kernel_size - 1) / 2)
```

```
    locs = []
```

```
    for i in range(num_keypoints):
```

```
        ind = np.unravel_index(np.argmax(f_Harris, axis=None),  
                               f_Harris.shape)
```

} → Vị trí của điểm có hệ số lớn nhất của mảng tại thời điểm hiện tại

```
        locs.append(ind)
```

```
        u, v = ind
```

```
        f_Harris[max(0, u-k) : min(f_Harris.shape[0], u+k+1),  
                  max(0, v-k) : min(f_Harris.shape[1], v+k+1)] = 0
```

} → Xóa dữ liệu trong vùng xung quanh điểm lớn nhất này

```
    locs = np.array(locs)
```

```
    return locs
```

Lecture 4: Programming exercise

Câu lệnh có sẵn trong opencv

```
dst = cv2.cornerHarris(img_gray,
```

```
    blockSize=5,
```

```
    ksize=3,
```

```
    k=0.08)
```

-----> Kích thước cửa sổ tính toán W

-----> Kích thước cửa sổ bộ lọc Sobel

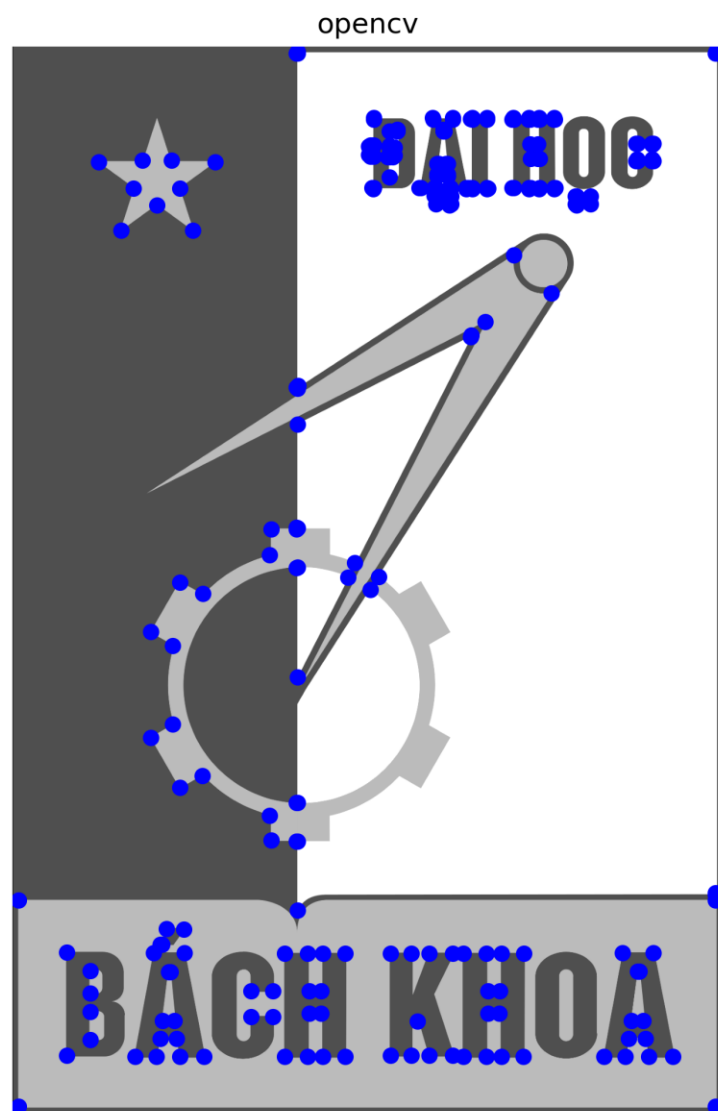
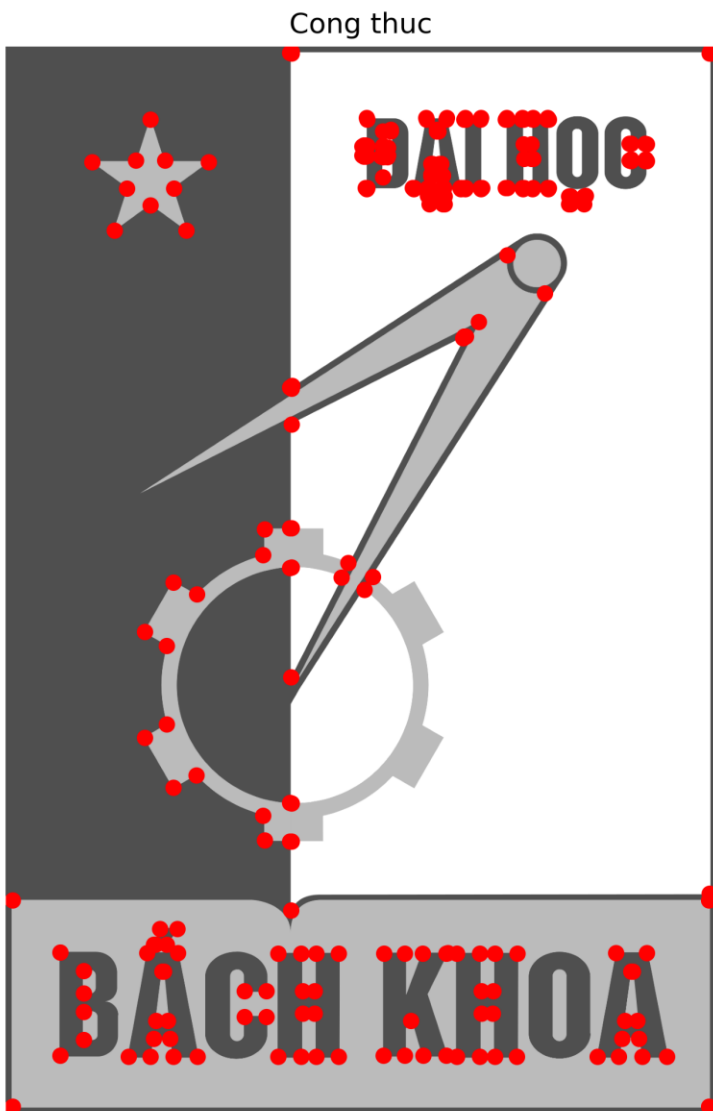
-----> Hệ số κ

```
cv_locs = NonMaximumSuppression(dst, 5, 250)
```

-----> Lựa chọn các điểm khả năng cao là góc

Lecture 4: Programming exercise

So sánh kết quả



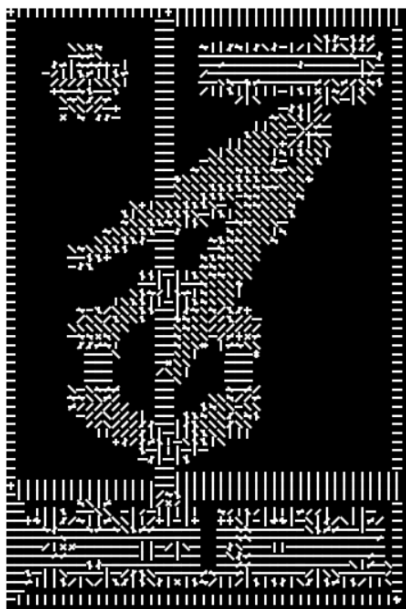
Lecture 4: Programming exercise

Hãy tìm góc trên hình sau (./data/C4_B1_a.png)



Lecture 4: Programming exercise

Bài 2: Cho ảnh như hình dưới đây (ảnh 'C4_B2.png' trong thư mục data đi kèm).



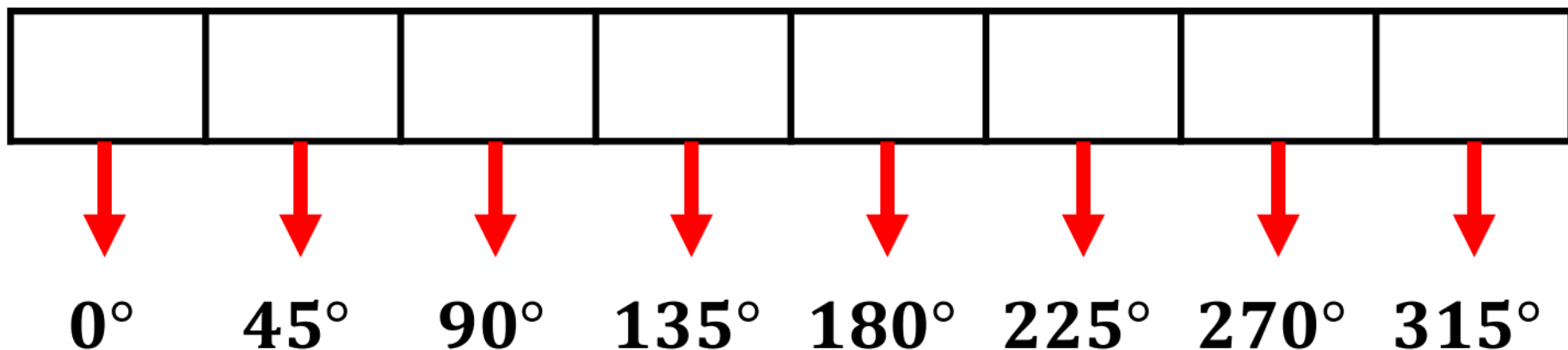
Lập trình tính vector đặc trưng HOG cho từng khu vực của ảnh có kích thước 4×4 . Sau đó biểu diễn vector này lên trên như hình minh họa.

Trong đó các góc hướng đại diện cho các giá trị của vector đặc trưng là: $0 - 45 - 90 - 135 - 180 - 225 - 270 - 315$.

Lecture 4: Programming exercise

Vector đặc trưng Histogram of Oriented Gradient

Vector đặc trưng của block ảnh



Lecture 4: Programming exercise

Vector đặc trưng Histogram of Oriented Gradient

Xét vị trí pixel có giá trị gradient là a và góc hướng là θ .

Giả sử: $\theta_i < \theta < \theta_{i+1}$, khi đó giá trị tại vị trí thứ i và $i+1$ được tăng thêm một lượng như sau:

$$\Delta a_i = \frac{\theta_{i+1} - \theta}{\theta_{i+1} - \theta_i} \times a$$

$$\Delta a_{i+1} = \frac{\theta - \theta_i}{\theta_{i+1} - \theta_i} \times a$$

Lecture 4: Programming exercise

Vector đặc trưng Histogram of Oriented Gradient

```
def CalculateHOGVector(A, Theta):
```

```
    HOG = np.zeros(8)
```

```
    for i in range(4):
```

```
        for j in range(4):
```

```
            if 0 <= Theta[i, j] and Theta[i, j] < 45:
```

```
                value1 = (45 - Theta[i, j]) / 45 * A[i, j]
```

```
                value2 = Theta[i, j] / 45 * A[i, j]
```

```
                HOG[0] += value1
```

```
                HOG[1] += value2
```

```
                pass
```

```
            ...
```

```
            else:
```

```
                value1 = (360 - Theta[i, j]) / 45 * A[i, j]
```

```
                value2 = (Theta[i, j] - 315) / 45 * A[i, j]
```

```
                HOG[7] += value1
```

```
                HOG[0] += value2
```

```
    HOG = HOG / np.linalg.norm(HOG)
```

```
    return HOG
```

Duyệt từng phần tử trong khối có kích thước 4×4

Tính toán cho trường hợp góc nằm trong khoảng $(0 - 45^\circ)$

Tính toán cho trường hợp góc nằm trong khoảng $(315 - 360^\circ)$

Chuẩn hóa vector HOG

Lecture 4: Programming exercise

Biểu diễn vector HOG trên ảnh

```
Angle = [0, 45, 90, 135, 180, 225, 270, 315]
```

```
Theta[Theta < 0] += 360
```

```
n_rows = int (img.shape[0] / 4)
```

```
n_cols = int (img.shape[1] / 4)
```

```
for i in range(n_rows):
```

```
    for j in range(n_cols):
```

```
        A_ij = A[i*4:(i+1)*4, j*4:(j+1)*4]
```

```
        Theta_ij = Theta[i*4:(i+1)*4, j*4:(j+1)*4]
```

```
        HOG_ij = CalculateHOGVector(A_ij, Theta_ij)
```

```
        x_vect = [0, 0]
```

```
        y_vect = [0, 0]
```

```
        for k in range(0, HOG_ij.shape[0]):
```

```
            if HOG_ij[k] > 0.5:
```

```
                x_vect[0] = 4*j+1.5+2*HOG_ij[k]*np.cos(Angle[k]*np.pi/180)
```

```
                x_vect[1] = 4*j+1.5-2*HOG_ij[k]*np.cos(Angle[k]*np.pi/180)
```

```
                y_vect[0] = 4*i+1.5+2*HOG_ij[k]*np.sin(Angle[k]*np.pi/180)
```

```
                y_vect[1] = 4*i+1.5-2*HOG_ij[k]*np.sin(Angle[k]*np.pi/180)
```

```
                plt.plot(x_vect, y_vect, color='white')
```

Tính số lượng vị trí tối đa của khối 4×4

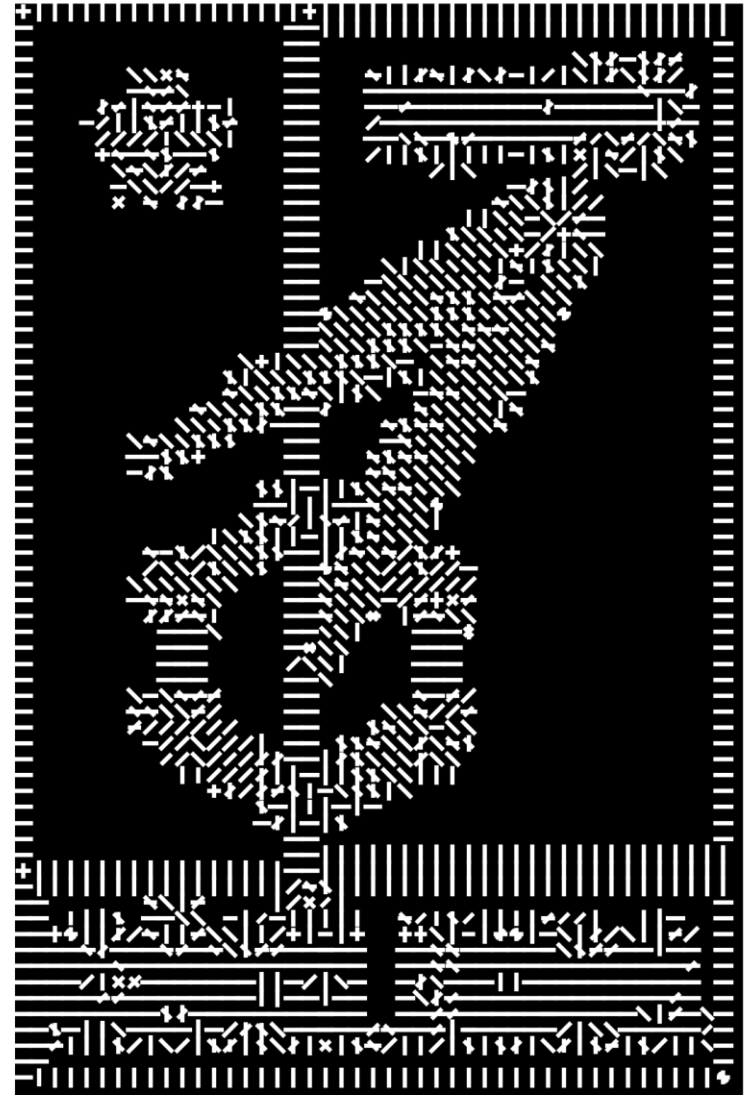
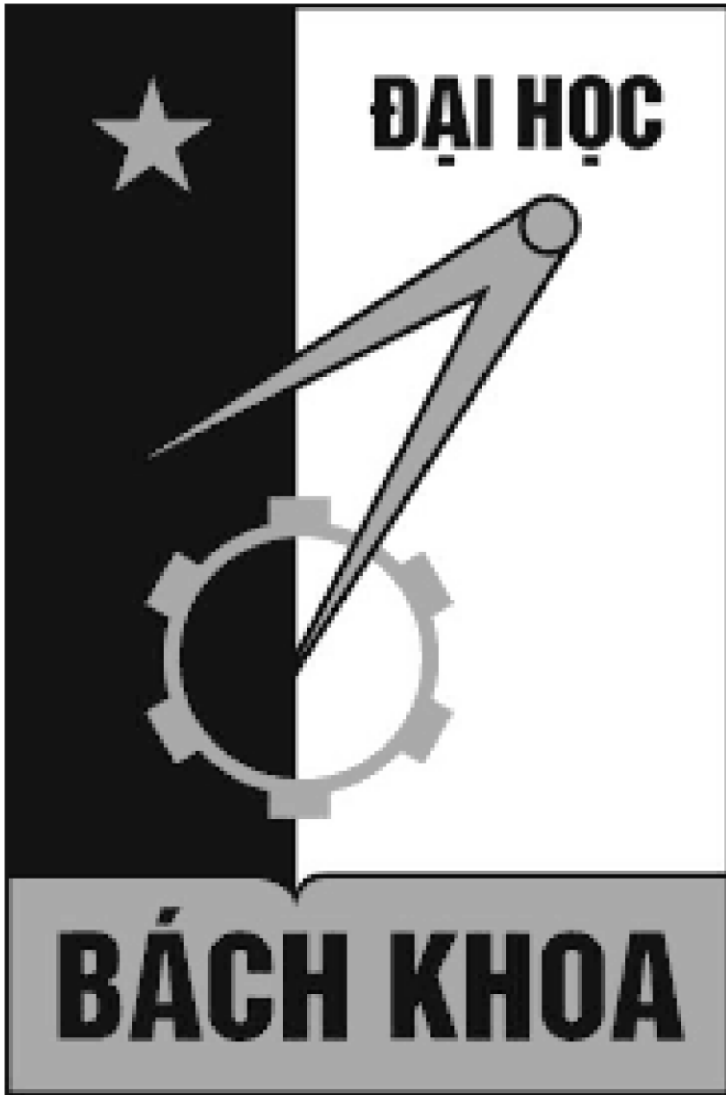
Tính vector HOG của khối 4×4 tương ứng

Chỉ biểu diễn hướng có ảnh hưởng lớn

Biểu diễn các hướng này

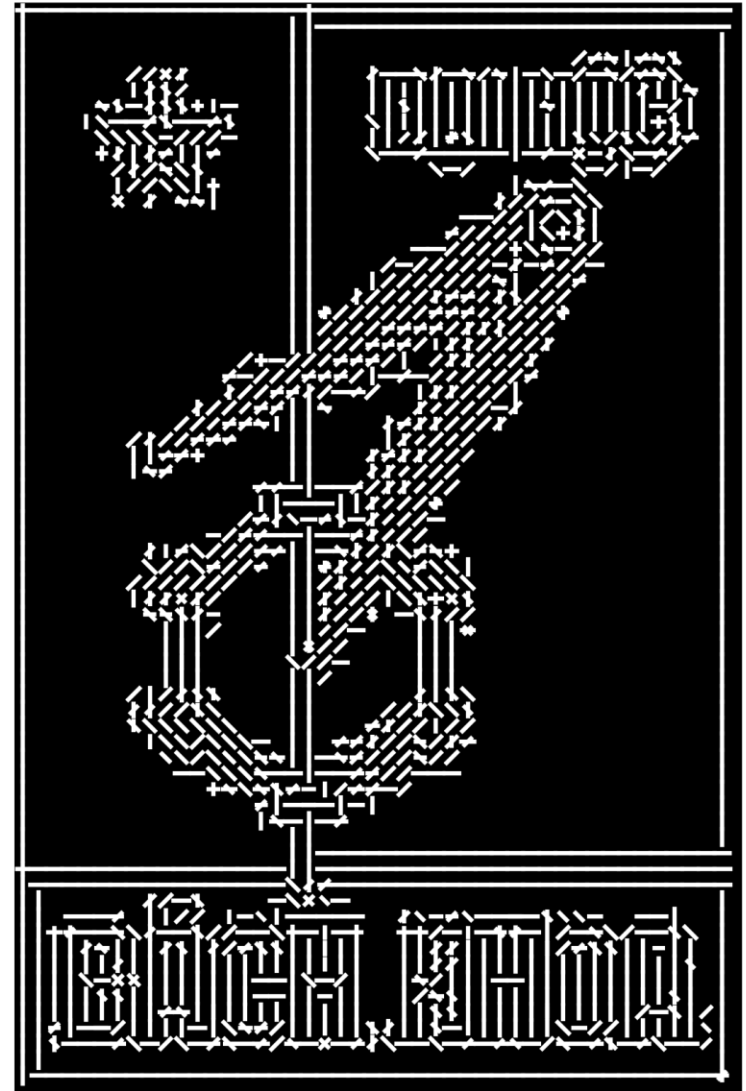
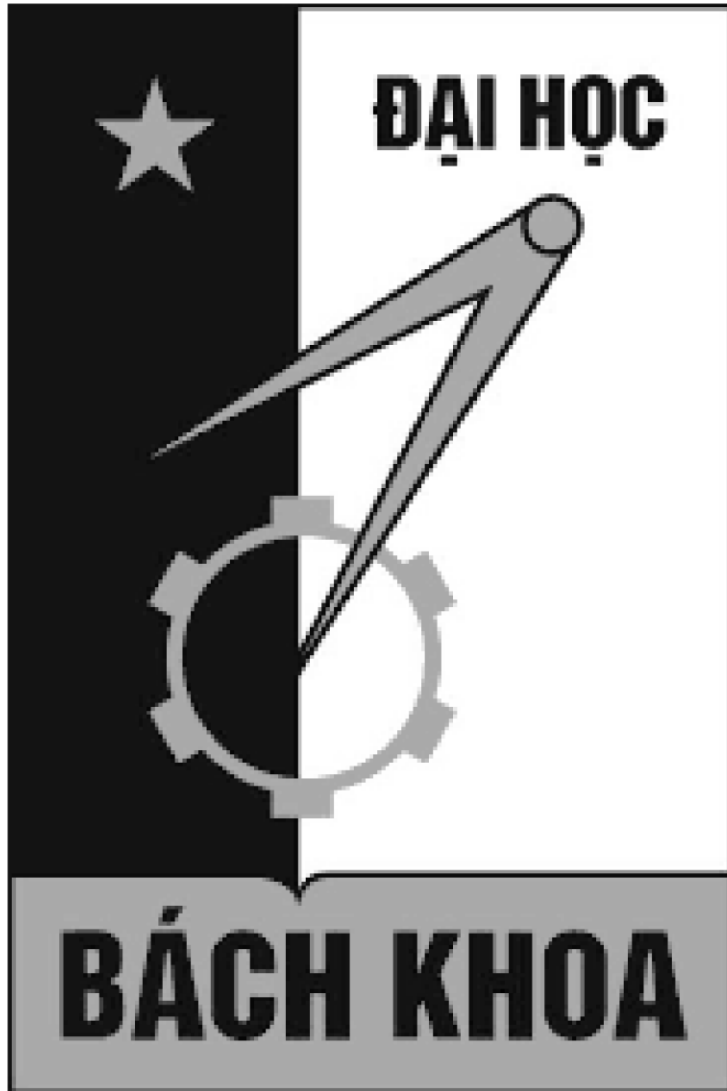
Lecture 4: Programming exercise

Biểu diễn vector HOG trên ảnh



Lecture 4: Programming exercise

Nếu cộng thêm một lượng 90 độ vào các hướng ???



Lecture 4: Programming exercise

Thay đổi các góc định hướng trong vector HOG thì kết quả thay đổi như thế nào?

?	?	?	?	?	?	?	?	?
0°	20°	40°	60°	80°	100°	120°	140°	160°